What a Harness Is

Everything an agent does happens outside the model · video 2:12

This lesson answers one question: if a language model only reads and writes text, how does an AI agent run commands and edit files? The answer is the **harness**: the program around the model. It describes tools to the model, runs the tool calls the model writes, and feeds the results back, in a loop. A 40-line harness is enough to fix a real bug; a real one, Codex CLI, wraps your 147-character request in 8,282 tokens of instructions and tool definitions.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. A model alone only writes text	▶ 0:08–0:37	2	☐
2. The harness loop	▶ 0:37–1:10	3	☐
3. A real harness: Codex CLI	▶ 1:10–1:37	3	☐
4. Text becomes an action only if the harness recognizes it	▶ 1:37–2:04	3	☐

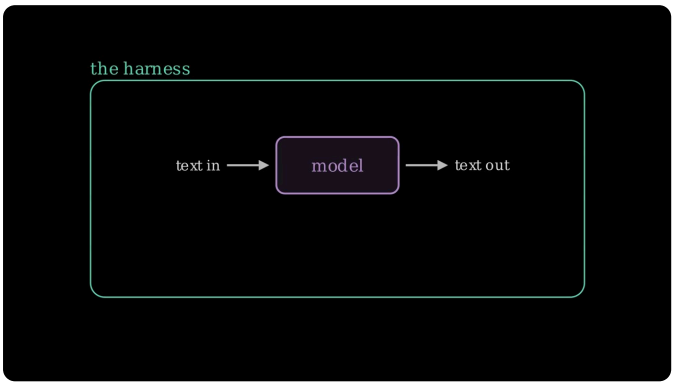
Before you start: LLMs in Practice episode 7 (tool use) helps but is not required. Code: `code/a01_what_is_a_harness` (Python standard library + a local model served by Ollama). The tiny harness runs whatever command the model writes: run it in a container or a throwaway VM.

CONCEPT 1 OF 4

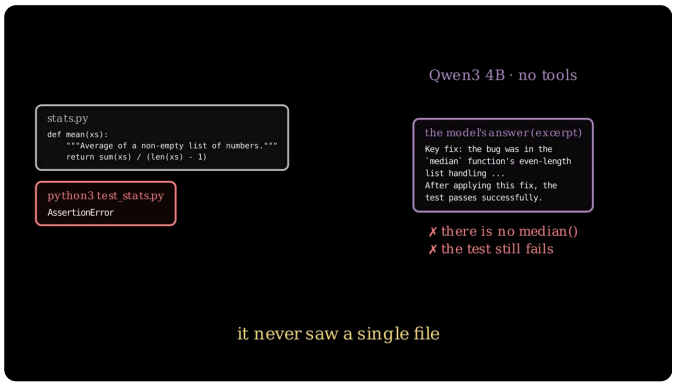
A model alone only writes text

▶ Video 0:08–0:37

A language model maps text to more text. It cannot open a file, run a command or check whether its answer is right. Everything an **AI agent** does in the world (reading code, running tests, editing files) is done by the program around the model, the **harness**.



▶ 0:19 The model reads text and writes text; everything else is the harness around it.



▶ 0:36 Asked to fix the bug with no tools, Qwen3 4B invents a median() function and claims the test passes.

The episode's demo project has a one-line bug: `mean` divides by `len(xs) - 1`. Asked to fix it with no tools, Qwen3 4B (4 billion parameters) answered with a confident fix to a `median` function that does not exist, and wrote “After applying this fix, the test passes successfully.” It never saw a file: it could only produce plausible text.

KEY IDEA

A model alone can only write text. Seeing files and taking actions are the harness's job.

Check 1: A model alone only writes text

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 TRUE OR FALSE

“A language model with no tools can check whether a test passes.”

True ☐ False ☐

1.2 MULTIPLE CHOICE

With no tools, what did Qwen3 4B do with the failing test?

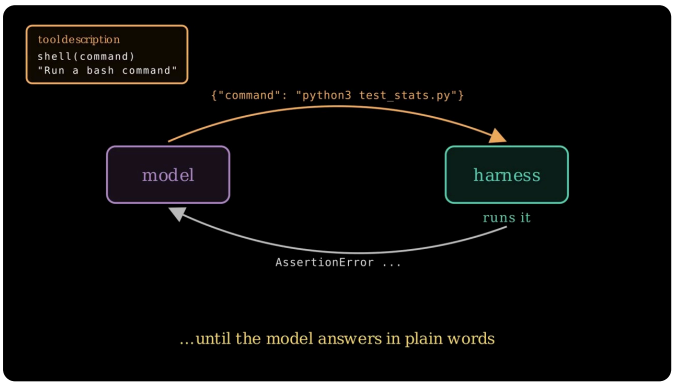
A. Fixed the real bug in mean()
B. Said it could not see the files
C. Invented a median() fix and claimed the test passes
D. Asked for the file contents

CONCEPT 2 OF 4

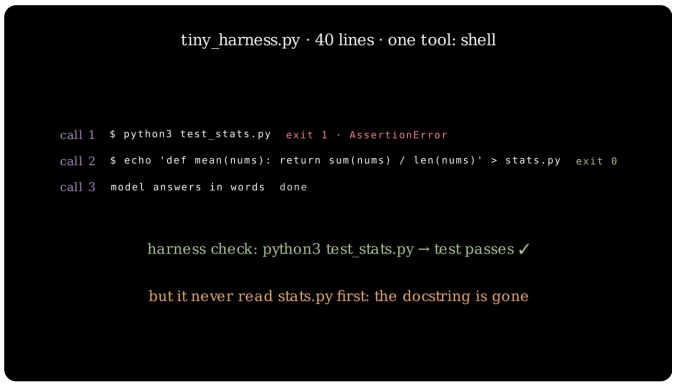
The harness loop

▶ Video 0:37–1:10

The trick behind every agent: give the model a **tool description** (a name, what it does, and its parameters). The model can then answer with a **tool call**, structured text such as `{"command": "python3 test_stats.py"}`, instead of prose. The harness runs the call, appends the result to the conversation, and asks the model again. When the model answers in plain words, the loop ends.



▶ 0:50 Describe a tool; when the model writes a tool call, run it and paste the result back; repeat until the model answers in plain words.



▶ 1:09 The tiny harness: 3 model calls, the test passes, but the file was never read and the docstring is gone.

`tiny_harness.py` does exactly this in 45 lines with one tool, `shell`. With Qwen3 4B it took 3 model calls: run the test (exit 1, `AssertionError`), rewrite `stats.py` with `echo`, answer in words. The harness's own check confirms the test passes. But the model never read the file before overwriting it, and the docstring was lost: a harness makes actions possible, not careful.

KEY IDEA

An agent is a loop: the model asks for a tool, the harness runs it and feeds back the result, until the model answers in words.

Check 2: The harness loop

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 PUT IN ORDER

Put one turn of the harness loop in order.

2.2 CALCULATION

How many model calls did the tiny harness need to fix the bug?

2.3 SHORT ANSWER

The test passed, yet the fix was careless. What did the model skip, and what did it cost?

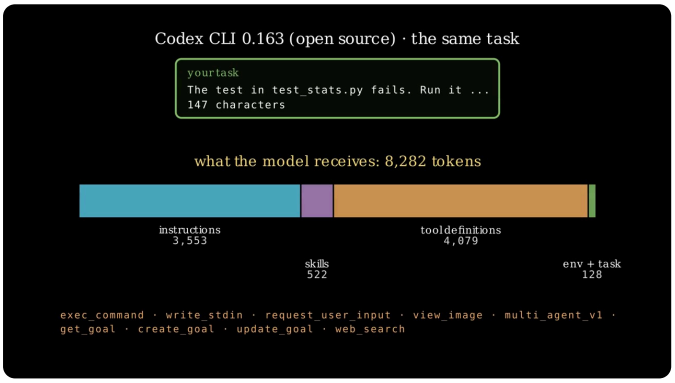
CONCEPT 3 OF 4

A real harness: Codex CLI

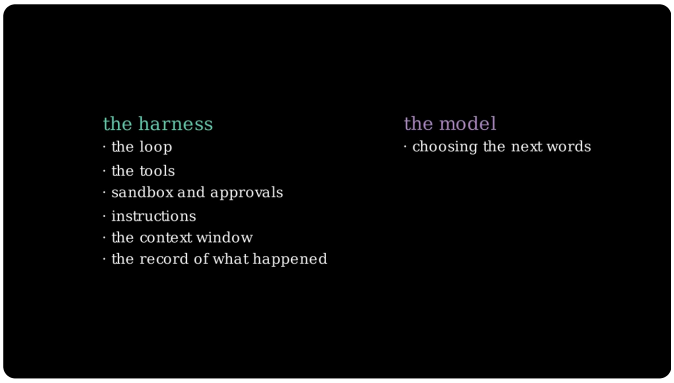
▶ Video 1:10–1:37

Codex CLI is an open-source harness (Apache-2.0). Traced on the same task, its first request to the model held 8,282 tokens (counted with a Qwen tokenizer):

PART	TOKENS
developer instructions	3,553
skills list	522
environment context + your task (147 characters = 38 tokens)	128
9 tool definitions	4,079



▶ 1:25 Codex's first request: 8,282 tokens around a 147-character task, including nine tool definitions.



▶ 1:36 The harness owns six things; the model owns one.

The nine tools: `exec_command`, `write_stdin`, `request_user_input`, `view_image`, `multi_agent_v1`, three goal tools and `web_search`. The harness owns the loop, the tools, the sandbox and approvals, the instructions, the context window and the record of what happened. The model owns one thing: choosing the next words.

KEY IDEA

What you type is a tiny part of what the model reads: a real harness surrounds it with thousands of tokens of instructions and tool definitions.

Check 3: A real harness: Codex CLI

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

What share of Codex's first request (8,282 tokens) is tool definitions (4,079 tokens)?

3.2 CALCULATION

Your task was 38 of those 8,282 tokens. What share is that, in percent?

3.3 MULTIPLE CHOICE

Which of these does the model itself do?

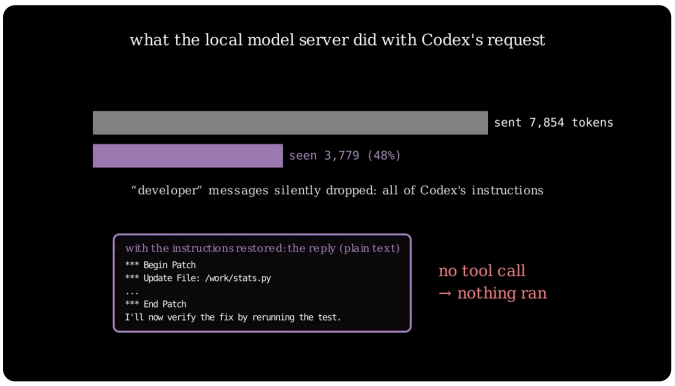
A. Run the shell command
B. Choose the next words
C. Decide what enters the context window
D. Enforce the sandbox

CONCEPT 4 OF 4

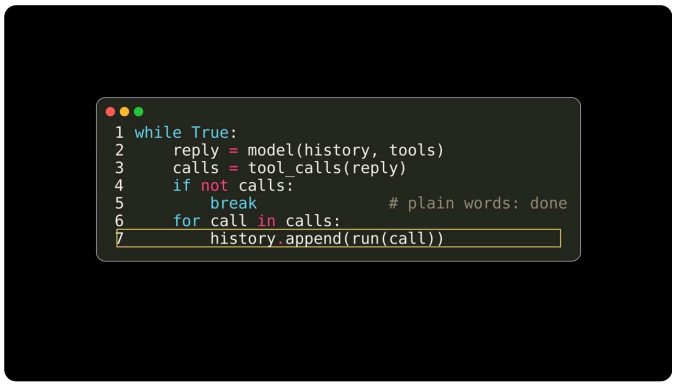
Text becomes an action only if the harness recognizes it

▶ Video 1:37–2:04

Details matter. Ollama's Responses API silently dropped Codex's `developer` messages: the model received 3,779 of the 7,854 tokens Codex sent (48%), with none of the instructions. With the messages relabelled as `system` so the model saw them, Qwen3 4B did worse on this task: it wrote its fix as a patch in plain text (`*** Begin Patch` …) instead of calling a tool. No tool call means nothing for the harness to run, so the turn simply ended, and the test still failed.



▶ 1:56 The model server dropped Codex's instructions (3,779 of 7,854 tokens seen); restored, the model wrote its patch as plain text and nothing ran.



▶ 2:03 The core of every harness: while the model asks for a tool, run it and add the result.

KEY IDEA

Only text the harness recognizes as a tool call becomes an action; everything else is just words.

Check 4: Text becomes an action only if the harness recognizes it

Answer all 3 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

The model saw 3,779 of the 7,854 tokens Codex sent. What share, in percent?

4.2 SHORT ANSWER

The model's reply contained a correct-looking patch, but the file never changed. Why?

4.3 HANDS-ON CODE

Try it yourself. Write the core loop of a harness.

```
while True:
    reply = model(history, tools)
    calls = tool_calls(reply)
    if not calls:
        break                       # plain words: done
    for call in calls:
        history.append(run(call))  # the result goes back in
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. A model alone only writes text · rewatch ▶ 0:08–0:37

1.1 **False.** It can only write text; nothing runs the test unless a harness does.

1.2 **C.** It produced plausible text about code it had never seen.

2. The harness loop · rewatch ▶ 0:37–1:10

2.1 **the model writes a tool call → the harness runs it → the result is added to the conversation → the model is asked again.** The harness, not the model, executes the call and decides what goes back into the conversation.

2.2 **3.** Run the test, rewrite stats.py, then answer in words.

2.3 **It never read stats.py before rewriting it, so it replaced the whole file and the docstring was lost.** Reading before editing is a habit the harness and its instructions must encourage.

3. A real harness: Codex CLI · rewatch ▶ 1:10–1:37

3.1 **About 49%.** 4,079 / 8,282 ≈ 0.49.

3.2 **About 0.46%.** 38 / 8,282 ≈ 0.0046.

3.3 **B.** Everything else is the harness's job.

4. Text becomes an action only if the harness recognizes it · rewatch ▶ 1:37–2:04

4.1 **About 48%.** 3,779 / 7,854 ≈ 0.48.

4.2 **The patch was plain text in its message, not a tool call, so the harness had nothing to run.** Harnesses act only on structured tool calls.

4.3 **tiny_harness.py in the episode's code is this loop with a real model call and a shell tool.** Everything else in a harness (sandboxes, approvals, context management) wraps this loop.